BLOCKCHAIN Experiment No: 2

Manas Patil
D20A/24

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Procedure : Stage 1: Blockchain Implementation Using Python in Google Colab

Step 1: Colab Setup and Library Installation

```
!pip install -q flask pyngrok

import hashlib
import json
import time
import requests
import flask
import pyngrok
import importlib.metadata

print("Environment setup completed successfully!")
flask_ver = importlib.metadata.version("flask")
print(f"Flask version: {flask_ver}")
print(f"pyngrok version: {pyngrok.__version__}")
```

Step 2 & 3: Complete Blockchain Implementation with Custom Requirements

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.pending_transactions = []
        self.create_genesis_block()

    def create_genesis_block(self):
        """Generates the first block of the chain with custom values and a valid hash."""
        genesis_block = {
            'index': 0,
            'timestamp': 1700000000.0, # fixed timestamp for reproducibility
            'transactions': [],
            'nonce': 0,
            'previous_hash': '0'
        }
        # Find a valid proof for the genesis block
        while not self.valid_proof(genesis_block):
            genesis_block['nonce'] += 1
        self.chain.append(genesis_block)

    @staticmethod
    def hash_block(block):
        """Computes the SHA-256 hash of a JSON-serialized block."""
        block_string = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

    def valid_proof(self, block):
        """Verifies if the block's hash meets the difficulty target (starts with '000')."""
        guess_hash = self.hash_block(block)
        return guess_hash.startswith('000')

    def proof_of_work(self, block_template):
        """Finds a golden nonce that produces a valid block hash starting with '000'."""
        block = block_template.copy()
        block['nonce'] = 0
        while not self.valid_proof(block):
            block['nonce'] += 1
        return block['nonce']

    def add_transaction(self, sender, receiver, amount):
        """Adds a new transaction to the pending list and returns the next block index."""
        self.pending_transactions.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        })
        return self.last_block['index'] + 1

    @property
    def last_block(self):
        return self.chain[-1]

    def mine_pending_transactions(self, miner_reward_address):
        """Bundles transactions, solves the PoW, creates a block, and appends it."""
        # Reward transaction included in this block
        reward_tx = {
            'sender': '0', # System sender for reward
            'receiver': miner_reward_address,
            'amount': 1.0
```

```
        }

        # Prepare the candidate transactions list
                        transactions_to_include =
self.pending_transactions.copy()
        transactions_to_include.append(reward_tx)

        new_block_template = {
            'index': len(self.chain),
            'timestamp': time.time(),
            'transactions': transactions_to_include,
            'nonce': 0,
                                    'previous_hash':
self.hash_block(self.last_block)
        }

        # Find golden nonce
                                        nonce =
self.proof_of_work(new_block_template)
        new_block_template['nonce'] = nonce

            # Append the new block and clear the
pending list
        self.chain.append(new_block_template)
        self.pending_transactions = []
        return new_block_template

    def is_chain_valid(self):
        """Validates the blockchain integrity."""
        for i in range(1, len(self.chain)):
            current_block = self.chain[i]
            previous_block = self.chain[i - 1]

            # Verify that previous_hash matches
                if current_block['previous_hash'] !=
self.hash_block(previous_block):
                return False

            # Verify Proof-of-Work condition
            if not self.valid_proof(current_block):
                return False

        return True

# Instantiate and verify basic structure
blockchain = Blockchain()
print(f"Blockchain initialized. Current length:
{len(blockchain.chain)}")
print(f"Genesis                         Block:
{json.dumps(blockchain.chain[0], indent=2)}")
print(f"Is                chain                valid?
{blockchain.is_chain_valid()}")
```

Step 4: Comprehensive Test Cases and Tamper Demonstration

```
# 1. Instantiate a fresh instance of the
Blockchain class
test_chain = Blockchain()
print(f"Initial                chain                length:
{len(test_chain.chain)}")
print(f"Initial                pending                transactions:
{test_chain.pending_transactions}\n")

# 2. Add several dummy transactions to verify
they populate pending_transactions
test_chain.add_transaction(sender="Alice",
receiver="Bob", amount=10.5)
test_chain.add_transaction(sender="Charlie",
receiver="Diana", amount=50.0)
```

```
print(f"Pending transactions after adding dummy TXs: {test_chain.pending_transactions}\n")

# 3. Mine a block to confirm details
print("Mining pending transactions...")
mined_block = test_chain.mine_pending_transactions(miner_reward_address="Miner_1")
print(f"Mined Block: {json.dumps(mined_block, indent=2)}")
print(f"Pending transactions queue after mining: {test_chain.pending_transactions}\n")

# 4. Verify initial blockchain validity
initial_validity = test_chain.is_chain_valid()
print(f"Is the blockchain valid initially? {initial_validity}\n")
assert initial_validity is True, "Blockchain should be valid before tampering!"

# 5. Simulate tampering by directly altering transaction details in block index 1
print("Simulating tampering... Modifying transaction amount in Block 1")
test_chain.chain[1]['transactions'][0]['amount'] = 9999.9
print(f"Modified Block 1 transactions: {test_chain.chain[1]['transactions']}\n")

# 6. Prove that the validation logic catches the tampering
post_tamper_validity = test_chain.is_chain_valid()
print(f"Is the blockchain valid after tampering? {post_tamper_validity}")
assert post_tamper_validity is False, "Blockchain validation failed to detect data tampering!"
print("Data tampering successfully detected by validation logic!")
```

Step 5: Flask API Development and Colab Tunneling

```
import threading
from flask import Flask, jsonify, request
import subprocess
import time

# Instantiate the Flask app
app = Flask(__name__)

# Instantiate a global Blockchain instance
api_blockchain = Blockchain()

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': api_blockchain.chain,
        'length': len(api_blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    values = request.get_json()
    if not values:
        return jsonify({'message': 'Missing transaction data'}), 400

    required = ['sender', 'receiver', 'amount']
    if not all(k in values for k in required):
        return jsonify({'message': 'Missing fields in
```

```
transaction payload'}), 400

    index = api_blockchain.add_transaction(
        sender=values['sender'],
        receiver=values['receiver'],
        amount=values['amount']
    )
    response = {'message': f'Transaction will be added to Block {index}'}
    return jsonify(response), 201

@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Default miner address for reward
    miner_address = "Miner_API_Node"
    new_block = api_blockchain.mine_pending_transactions(miner_reward_address=miner_address)
    response = {
        'message': 'New block mined successfully!',
        'block': new_block
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid_chain = api_blockchain.is_chain_valid()
    if is_valid_chain:
        response = {'message': 'The Blockchain is valid.', 'is_valid': True}
    else:
        response = {'message': 'Warning: The Blockchain is invalid!', 'is_valid': False}
    return jsonify(response), 200

def run_flask():
    # Run Flask on port 5005 to avoid Address already in use error
    app.run(host='0.0.0.0', port=5005, debug=False, use_reloader=False)

# Start Flask in a background thread
flask_thread = threading.Thread(target=run_flask)
flask_thread.daemon = True
flask_thread.start()
print("Flask application started in background thread on port 5005!")

# Establish localtunnel for external access as alternative to ngrok
def start_localtunnel():
    time.sleep(2)
    print("Starting localtunnel...")
    lt_proc = subprocess.Popen(['npx', 'local-tunnel', '--port', '5005'], stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)
    for line in lt_proc.stdout:
        if "your url is" in line:
            print(line.strip())
            break

lt_thread = threading.Thread(target=start_localtunnel)
lt_thread.daemon = True
lt_thread.start()
```

Step 6 : Save and Download as blockchain.py : Save and Download as blockchain.py

Stage 2: Flask API Development, Ngrok Tunneling and Postman Integration

Step 1: Install Required Libraries

!pip install flask pyngrok

- Flask: Used to create and run the blockchain web application.
- Pyngrok: Used to expose the local Flask server through a public URL.

Step 2: Upload the Python File

from google.colab import filesuploaded = files.upload()

Step 3: Import the Blockchain Module

import blockchainprint("Blockchain module imported successfully!")

Output:

Blockchain module imported successfully!

Step 4: Verify the Flask Application

print(blockchain.app)

Output:

<Flask 'blockchain'>

This confirmed that the Flask application was available.

Step 5: Start the Flask Server

from threading import Threadflask_thread = Thread(target=lambda: blockchain.app.run(host="0.0.0.0", port=5000, use_reloader=False), daemon=True)flask_thread.start()

The Flask application started running on port 5000.

Step 6: Verify the Flask Server

We checked whether the Flask server was responding successfully by sending a GET request to the /get_chain endpoint.

import requeststry: response = requests.get("http://127.0.0.1:5000/get_chain") print("Status Code:", response.status_code) print("Response:", response.json())except Exception as e: print("Error:", e)

Output:

- Status Code: 200
- The blockchain data was successfully retrieved in JSON format.

Step 7: Configure Ngrok Authentication

We created an account on the ngrok website and obtained an authentication token.

Website: https://ngrok.com/

We configured the authentication token in Google Colab.

from pyngrok import ngrokngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")

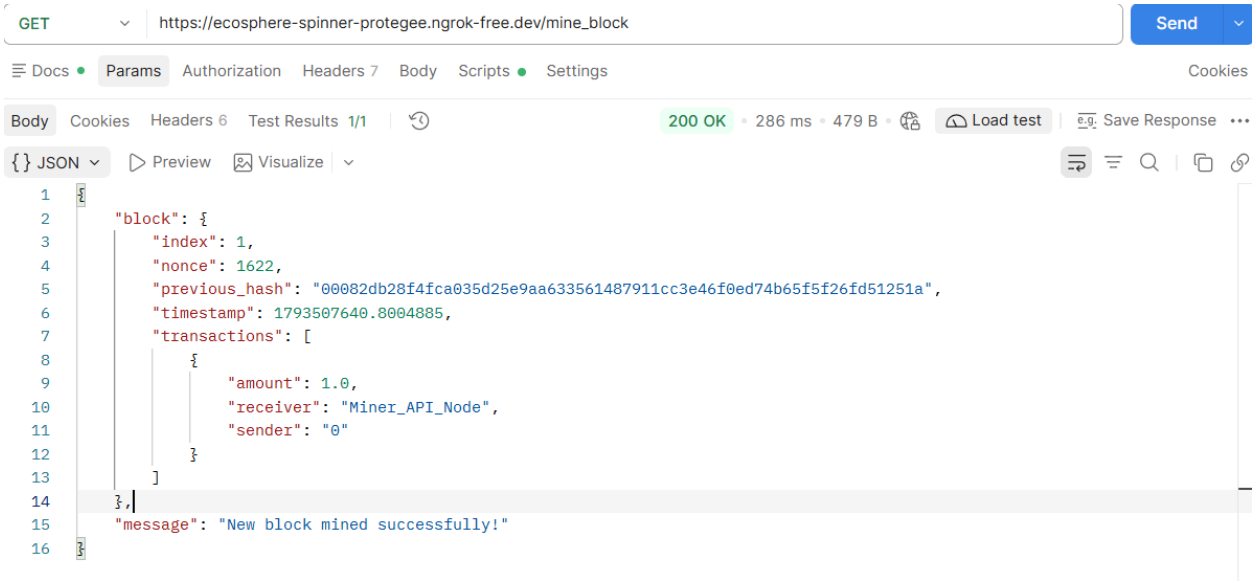
Step 8: Generate a Public URL

We used ngrok to expose the Flask application running on port 5000.

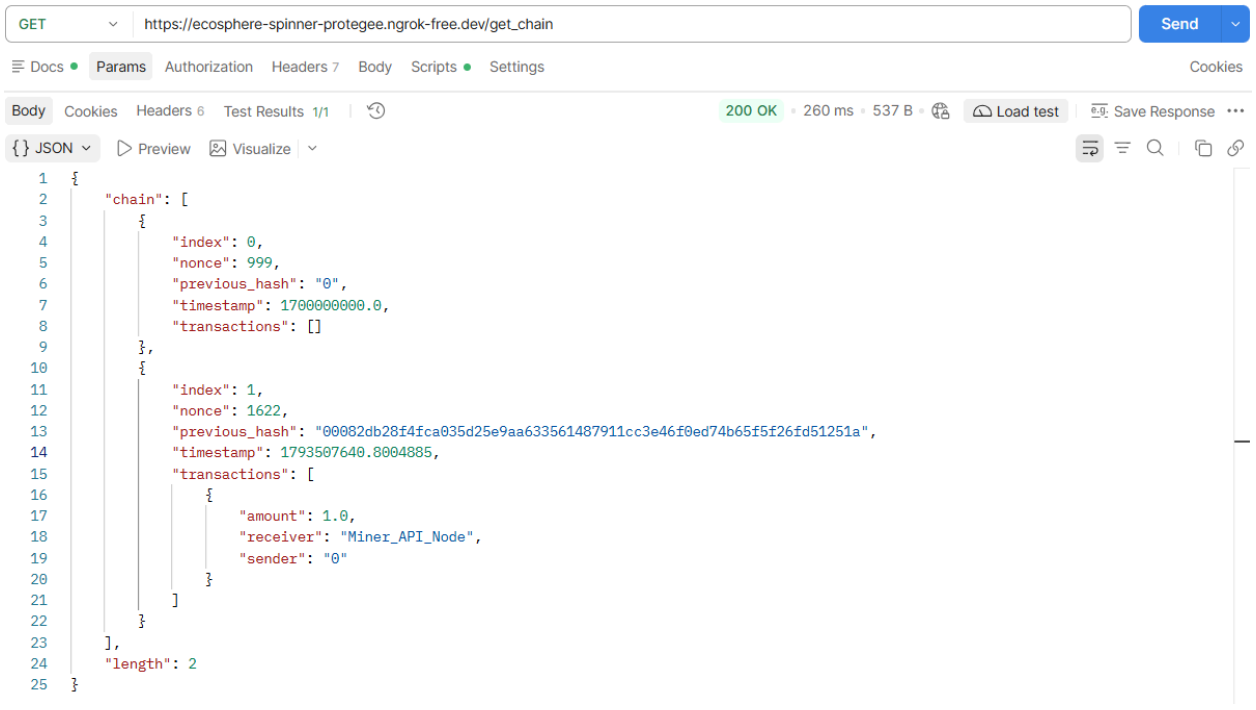
public_url = ngrok.connect(5000)print("Public URL:", public_url)
A public URL was generated, which allowed Postman to communicate with the Flask application running in Google Colab.

Step 9: Open Postman

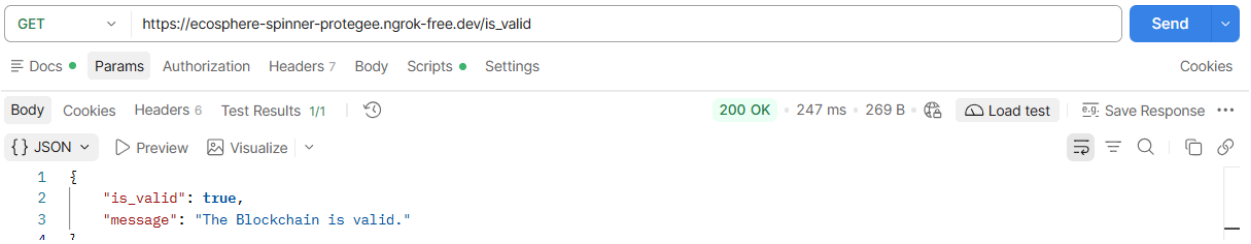
Test 1: Mine a block



Test 2: Display the blockchain



Test 3: Validate the blockchain



Conclusion :

We successfully implemented a basic blockchain using Python in Google Colab. We created blocks and transactions, implemented SHA-256 cryptographic hashing, and applied the Proof-of-Work mechanism to find a valid nonce satisfying the condition of three leading zeros (000). We also implemented blockchain validation to verify the integrity of the chain. Furthermore, we integrated the blockchain with Flask, exposed the application using ngrok, and tested the /mine_block, /get_chain, and /is_valid API endpoints using Postman. Thus, we successfully demonstrated the fundamental concepts of blockchain, mining, cryptographic hashing, and blockchain validation.